FitFusion AI

Live Demonstration Guide: Model 1 (Session Energy Expenditure) and Model 2 (Meal Recommender)

Project	FitFusion AI: Next.js web app + Python FastAPI ML service
Model 1	`POST /predict/calories` : calibrated Keytel heart-rate equation with MET fallback
Model 2	`POST /recommend` : popularity-ranked Food.com recipes inside dietary and calorie filters
Demo command	`.venv\Scripts\python.exe demo_models.py` (run from `ml-service`)
Prepared	17 September 2026, from the project source code and real run output

Contents

1. Read this first: what the code does and does not support
2. Model 1: Session energy expenditure (sections 1 to 10)
3. Model 2: Meal recommender (sections 1 to 10)
4. Complete end-to-end demo flow
5. Presentation script
6. Pre-presentation checklist

Read this first: what the code does and does not support

1. **The project is FitFusion, not "Face Fusion".** Nothing in either model touches faces or images. Both work on numbers (heart rate, body stats) and recipe data. If asked how this differs from image processing, the honest answer is that it is not image processing at all.
2. **Model 1's output does not feed Model 2.** They are two separate branches that start from the same user profile. Model 2's calorie targets come from the Mifflin-St Jeor formula in `web/src/lib/meal-targets.ts`, which states that logged workouts are not added on top. The end-to-end flow in this guide matches the real code.
3. **Neither shipped model is a trained ML model when it runs.** Model 1 is a published heart-rate equation plus a 2-parameter linear calibration fitted on lab data. Model 2 ranks recipes by popularity inside filters. In both cases a trained model (XGBoost for Model 1, ALS for Model 2) was built, evaluated, and lost to or tied the simpler method, so it was not shipped. Present this as **data-driven model selection**.
4. **Model 2 does not personalise per user.** `user_id` is accepted in the request but never used in `recommend.py`. Users with the same restrictions and calorie targets get the same recipes.

MODEL 1 Session Energy Expenditure

`POST /predict/calories` · `ml-service/app/routers/calorie.py`

1. Purpose of the model

- **Problem:** estimate how many calories a user burned in a logged workout.
- **Why it is needed:** the app previously used only a MET table (activity type × weight × time), which ignores how hard the person actually worked. Heart rate measures effort directly.
- **Role in the pipeline:** when a user logs a cardio session on the Progress page, the calorie value saved to `cardio_logs.calories_burned` comes from this model.

2. Model architecture / working

- **Type:** the Keytel et al. (2005) heart-rate equation, followed by a linear calibration `kcal/min = 0.1091 + 0.5579 × Keytel`. The two parameters were fitted on WEEE lab calorimetry data (16 participants, 40 exercise segments). A MET-table fallback covers every other case.
- **Libraries:** FastAPI and Pydantic (serving, validation), NumPy. Training and evaluation used XGBoost, scikit-learn (Ridge), SHAP, SciPy (Wilcoxon test) and matplotlib.
- **Why calibrate:** on the WEEE lab data, uncalibrated Keytel over-estimated by +3.82 kcal/min on average.
- **Why not XGBoost:** an XGBoost model was trained on the Malaga treadmill dataset (PhysioNet, 857 people). It beat raw Keytel, but once both were calibrated they tied (MAE 1.27 vs 1.29 kcal/min, Wilcoxon p = 0.16). The simpler method shipped. The trained model remains in `models/calorie_hr_model.pkl` but serving does not load it.

Processing steps (input → output)

1. Pydantic validates the request, converts `duration_min` ↔ `duration_hours`, maps "Male"/"Female" to `male` / `female`, and normalises the workout type.
2. Five checks run. If any fails, the MET estimate is returned with a `reason`: calibration not loaded; no heart rate; not Cardio; age or sex missing; heart rate outside 83.5–182.7 bpm.
3. The Keytel equation gives kJ/min, divided by 4.184 to get kcal/min.
4. Calibration: `max(0, 0.1091 + 0.5579 × raw)`.
5. The per-minute value is multiplied by minutes. The MET estimate is returned alongside for comparison.

3. Input

JSON body, with `Authorization: Bearer <ML_SERVICE_API_KEY>` header.

Field	Type	Rule
`weight_kg`	float	required, > 0
`age`	float	optional, 10–100 (needed for the heart-rate method)
`sex`	string	"male"/"female" (also accepts "Male", "m", ...); anything else is treated as unknown
`avg_hr`	float	optional, 30–230 bpm
`duration_min` or `duration_hours`	float	at least one required

`workout_type`	string	Cardio / Strength / Yoga / HIIT
`intensity`	string	low / moderate / high (used only by the MET fallback)

Example:

```
{"weight_kg": 70, "age": 22, "sex": "Male", "avg_hr": 150, "duration_min": 30, "workout_type":
"Cardio"}
```

> *"The input to this model is a workout session (average heart rate, duration and workout type) plus the user's age, sex and weight. It is validated by a Pydantic schema, which checks ranges and normalises units, before being passed to the model."*

4. Output

```
{"calories": 234.9, "kcal_per_min": 7.83, "method": "heart_rate", "validated": true,
 "expected_error_kcal_per_min": 1.29, "reason": null, "baseline_met_calories": 262.5,
 "model_version": "energy-keytel-cal-v1", "workout_type": "Cardio", "stub": false}
```

- `method`: `heart_rate` or `met_table`. `reason` explains why the fallback was used.
- `expected_error_kcal_per_min`: the error measured on held-out lab participants.
- **Next stage:** `CardioTab.tsx` (line 222) saves `estimate.calories` when `method === "heart_rate"`, otherwise its own MET value. The call goes through `/api/workout-log`, which has a fallback if the Python service is down.

> *"The output is the session's calories and calories per minute, with the method used and its expected error. This output is saved as the calories burned in the user's cardio log."*

5. Files needed to run

What	File(s)
Script to run	`ml-service/demo_models.py` (or the server `app/main.py`)
Model code	`app/energy.py`, `app/routers/calorie.py`, `app/schemas.py`, `app/model_registry.py`, `app/auth.py`
Artifact	`ml-service/models/calorie_energy.json` (intercept, slope, HR range, expected error)
Configuration	`ml-service/.env` with `ML_SERVICE_API_KEY` (do not show on screen)
Evidence	`models/reports/calorie_hr_calibration.json`, `calorie_hr_calibration.png`, `calorie_hr.json`
Dependencies	existing `ml-service/.venv` (from `requirements.txt`)
Training data (offline only)	`ml-service/data/malaga`, `ml-service/data/weee`

Command (PowerShell, inside `ml-service`):

```
.venv\Scripts\python.exe demo_models.py 1
```

Retraining (do not run live): `.venv\Scripts\python.exe -m training.train_calorie_hr`, then `-m training.calibrate_calorie_hr`.

6. Live demonstration procedure

Step	What to do and show
1. Open the project	Open VS Code at `fitfusion-ai-wb`. Show the `ml-service/` folder. Say: "the web app calls this Python service."
2. Open the model code	`app/energy.py` line 22: the Keytel equation. Then `app/routers/calorie.py` lines 64–108: the five checks, and the calibration at lines 94–95.
3. Show the parameters	Open `models/calorie_energy.json`. Point to `intercept`, `slope`, `validated_hr_range_bpm`, `expected_mae_kcal_per_min`.
4. Run the model	`cd ml-service` then `.venv\Scripts\python.exe demo_models.py 1`
5. Show the output	Point to the PROCESSING lines (13.84 → 7.83 → 234.9) and `method: heart_rate`.
6. Explain the result	Walk through the arithmetic, then the fallback cases and their reasons. Open `calorie_hr_calibration.png`: the uncalibrated panel sits above the diagonal; the calibrated panels sit on it.

7. What code to show

Where	What it does	Why it matters
`app/energy.py` 22–35	Keytel male and female equations	The core formula
`app/routers/calorie.py` 73–92	Validity checks → MET fallback with a reason	The model is used only where it was validated
`app/routers/calorie.py` 94–95	Calibration `intercept + slope × raw`	The part learned from lab data
`app/schemas.py` 35–72	Input validation and normalisation	Invalid input is rejected with HTTP 422
`training/calibrate_calorie_hr.py` docstring	Leave-one-participant-out design and decision rule	Shows how the method was chosen

8. Expected demonstration output

```
Service loaded: calorie=True  recommender=True

MODEL 1 - session energy expenditure   POST /predict/calories
--- PROCESSING ---
1. Keytel (2005) male equation -> 13.84 kcal/min (uncalibrated)
2. Calibration 0.1091 + 0.5579 x 13.84 = 7.83 kcal/min
3. x 30 minutes = 234.9 kcal

--- OUTPUT (HTTP 200) ---
  "calories": 234.9, "kcal_per_min": 7.83, "method": "heart_rate", ...

--- FALLBACK CASES ---
  no heart rate               ->  262.5 kcal  method=met_table  reason=no average heart rate
supplied
  strength session            ->  175.0 kcal  method=met_table  reason=heart-rate method is
validated for cardio only, not Strength
  HR outside validated range  ->  262.5 kcal  method=met_table  reason=average heart rate 200 bpm
is outside the validated range 83.5-182.7 bpm
  invalid input (weight -5)   -> HTTP 422: Input should be greater than 0
  no bearer token             -> HTTP 401: Missing bearer token
```

How to verify it worked: HTTP 200, `stub: false`, `model_version: energy-keytel-cal-v1`. If `stub: true` appears, `calorie_energy.json` did not load.

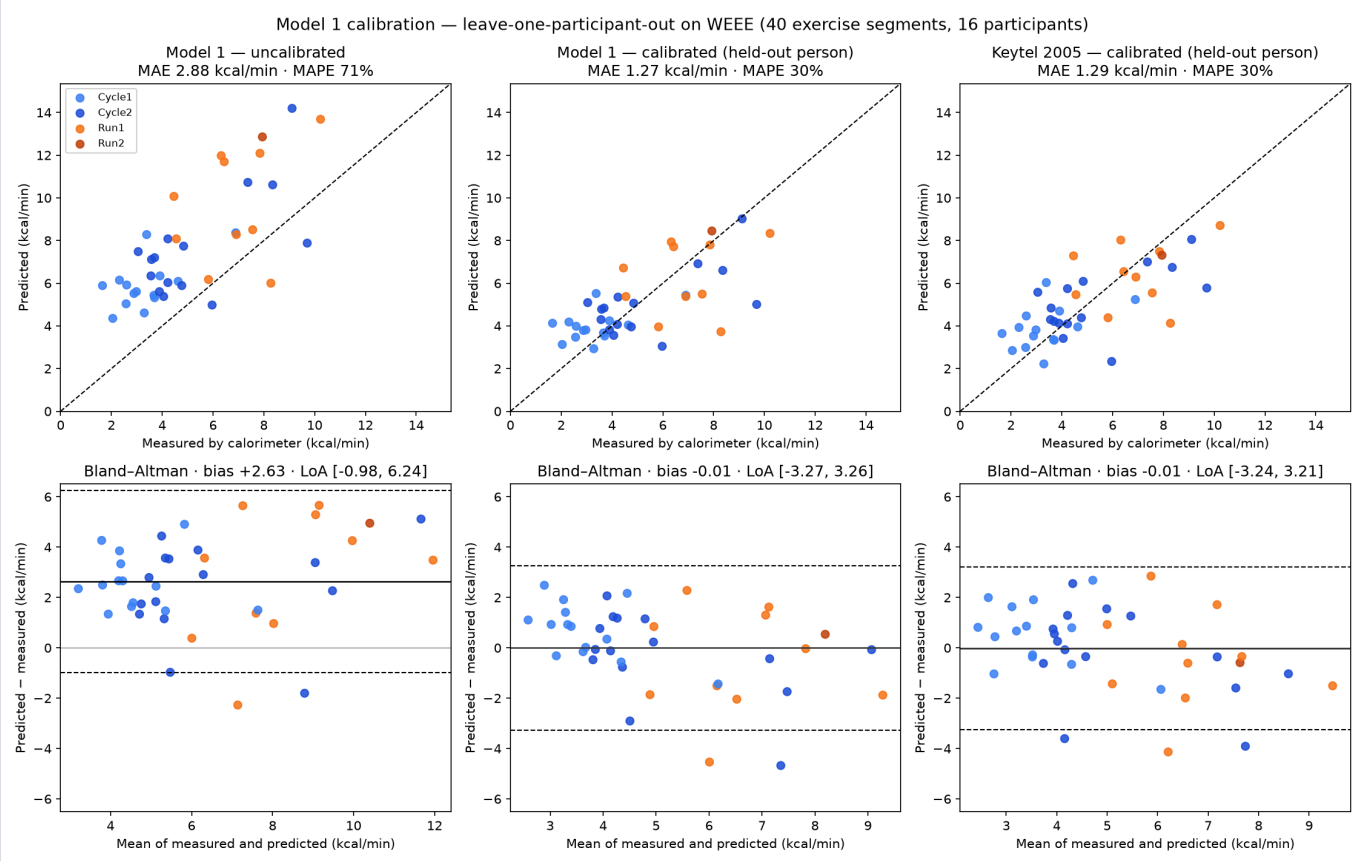


Figure 1: `models/reports/calorie_hr_calibration.png`. Top row: predicted vs measured kcal/min (the dashed line is perfect agreement). Bottom row: Bland-Altman plots. Left: uncalibrated (MAE 2.88). Middle and right: calibrated, held-out participants (MAE 1.27 and 1.29).

9. Viva explanation

30 seconds

"Model 1 estimates the calories burned in a cardio session from heart rate. It uses the published Keytel equation, corrected with a linear calibration I fitted on lab calorimetry data. If heart rate is missing, or the session is not cardio, it falls back to a MET table and says why."

1 minute

"Model 1 estimates the calories burned in a cardio session from heart rate, using the published Keytel equation with a linear calibration I fitted on lab calorimetry data. I also trained an XGBoost model on 857 people's treadmill tests. It beat the raw equation, but once both were calibrated they tied: 1.27 versus 1.29 kcal per minute error, p = 0.16 on held-out participants. So I shipped the simpler, explainable method. The expected error is about 1.3 kcal per minute, roughly 30%, and the app reports it with every answer. When heart rate, age or sex is missing, or the workout is not cardio, it returns a MET estimate and the reason."

Technical (if asked deeper)

- **Training data:** Malaga (PhysioNet treadmill-exercise-cardioresp v1.0.1). Labels from the Weir equation on breath-by-breath VO_2/VCO_2, windowed into 60-second segments.
- **Candidates:** Ridge and monotone XGBoost (energy may only rise with HR and weight), selected by 5-fold cross-validation grouped by person.
- **External test:** WEEE (Zenodo 10.5281/zenodo.6420886), label $VO_2 \times 5.0$ kcal/L.
- **Calibration evaluation:** leave-one-participant-out (16 folds). Metrics: MAE, MAPE, bias, Bland-Altman limits of agreement, R^2, paired Wilcoxon test.
- **SHAP:** heart rate dominates (mean |SHAP| 3.24, every other feature ≤ 0.52).

10. Likely questions from teachers

Question	Answer
Why did you use this model?	On held-out people, a trained XGBoost model was no more accurate than the calibrated equation, and the equation is transparent and explainable.
What is the input?	Average heart rate, duration, workout type, age, sex, weight (JSON).
What is the output?	Calories, kcal/min, method used, expected error, and a reason when the fallback is used.
Why this architecture?	Only 16 calibration participants: too little data for a complex model. A 2-parameter linear correction cannot overfit, and it matched the ML model's accuracy.
What preprocessing is performed?	Serving: Pydantic range checks, unit conversion, sex and workout-type normalisation. Training: HR 40–220 filter, drop the first 60 s, exercise phase only, RER ≤ 1.0, 60-second windows.
What is it learning?	Two calibration parameters (intercept 0.1091, slope 0.5579) that correct Keytel's over-estimate on the WEEE population.
How is it evaluated? Metrics?	Leave-one-participant-out: MAE 1.29 kcal/min, MAPE 30%, R^2 0.46, Bland-Altman limits of agreement −3.24 to +3.21. For a 30-minute session: about 39 kcal error vs about 51 for the MET table.

Limitations?	Only 16 calibration participants; validated for cycling and treadmill cardio only; ~30% error per session; strength and yoga use the MET table. The shipped parameters are refitted on all 16 people, so the expected error is the cross-validated estimate.
How is the output passed on?	Returned to `/api/workout-log` → Cardio tab → saved as `cardio_logs.calories_burned`.
What if the input is invalid?	Out-of-range fields → HTTP 422. Missing HR/age/sex or out-of-range HR → MET estimate plus `reason`. Service down → Next.js uses its own MET value; the user sees no error.
How is it different from simple image processing / a simple formula?	No images are involved. Compared with a plain formula: the correction is fitted on real calorimetry measurements and its error is measured on people it never saw.

MODEL 2 Meal Recommender

`POST /recommend` · `ml-service/app/routers/recommend.py`

1. Purpose of the model

- **Problem:** the diet plan used to come from an LLM, which invented meals and calorie counts.
- **Why it is needed:** meals must be real recipes, with real calories, that respect dietary restrictions.
- **Role in the pipeline:** during plan generation it supplies candidate recipes for each meal slot. The LLM must choose from them, and code enforces that.

2. Model architecture / working

- **Type:** not ML at serving time. Recipes are ranked by popularity (likes in the last 365 days of the data, which ends 20 Dec 2018) inside rule-based filters.
- **Why not ML:** an implicit ALS collaborative filter was trained and compared in a rolling yearly evaluation (2012–2017, 20,031 user-years). Popularity scored higher: Recall@10 0.0197 vs 0.0164; the 95% CI of the difference, [−0.0051, −0.0013], lies entirely below zero. Decision: DROP ALS.
- **Libraries:** FastAPI, Pydantic, pandas, joblib. Evaluation: `implicit` (ALS), matplotlib.

Offline catalog build (`training/build_meal_catalog.py`)

1. Food.com dataset: 231,637 recipes and 1.13M ratings. A rating of 4–5 stars counts as a "like".
2. Keep recipes with 20–2000 kcal per serving, a meal course tag, not a sauce or condiment, and at least one like. Result: 98,325 recipes.
3. Dietary flags computed from ingredient lists, with the recipe name as a second check (`training/data_foodcom.py`).
4. Sort by likes in the last year, then all-time likes. Save as `models/meal_catalog.pkl`.

Per request

1. Normalise restrictions. If any is unsupported (halal, kosher, ...), return no recipes plus a reason.
2. For each slot, filter by course tag (lunch and dinner exclude desserts and breads), dietary flags, protein floor, and already-used recipes.
3. Keep recipes within ±25% of the slot's calorie target; if too few, widen to ±50%.
4. Take the top N by popularity. A recipe chosen for one slot is not repeated in later slots.

3. Input

Field	Type	Rule
`meals`	list of `{slot, kcal_target, min_protein_g}`	1–6 items; slot ∈ breakfast / lunch / dinner / snack; kcal_target > 0 and ≤ 3000
`dietary_restrictions`	list of strings	e.g. `["vegetarian"]`
`per_meal`	int	1–20 (default 5)
`exclude_recipe_ids`	list of int	optional

Preprocessing before the model (Next.js, `web/src/lib/meal-targets.ts`): BMR (Mifflin-St Jeor) × activity factor; −20% for weight loss or +10% for muscle gain; split 25 / 35 / 30 / 10% across breakfast / lunch / dinner /

snack; protein floors only for muscle gain. If age, height or weight is missing, no targets are produced and the recommendation step is skipped.

Example:

```
{"meals": [{"slot": "breakfast", "kcal_target": 524, "min_protein_g": 0},
           {"slot": "lunch",     "kcal_target": 733, "min_protein_g": 0},
           {"slot": "dinner",    "kcal_target": 628, "min_protein_g": 0},
           {"slot": "snack",     "kcal_target": 209, "min_protein_g": 0}],
 "dietary_restrictions": ["vegetarian"], "per_meal": 3}
```

> *"The input to this model is a calorie target for each meal slot plus the user's dietary restrictions. The targets are computed from the Mifflin-St Jeor formula in Next.js before being passed to the recommender."*

4. Output

- For each slot: `kcal_min`, `kcal_max`, `widened`, and `recipes[]`. Each recipe has `recipe_id`, `name`, `kcal`, `protein_g`, `minutes`, `likes_last_year`, `likes_all_time`.
- Also: `applied_restrictions`, `unsupported_restrictions`, `reason`, `method`, `model_version`, `stub`.
- **Next stage:** `/api/generate-plan` calls it (route.ts line 956), puts the recipes into the Groq LLaMA prompt, then `enforceRecipeMeals` (line 680) replaces any meal that is not one of the recommended recipes with the top recipe. Method and version are recorded in `ml_meta.meal_recommender`.

> *"The output is a ranked list of real recipes for each meal, with their calories and protein. This output is used to build the user's diet plan, and the code guarantees every meal in the plan is one of these recipes."*

5. Files needed to run

What	File(s)
Script to run	`ml-service/demo_models.py`
Model code	`app/routers/recommend.py`, `app/schemas.py`, `app/model_registry.py`; `training/data_foodcom.py` (dietary flags), `training/build_meal_catalog.py`
Artifact	`ml-service/models/meal_catalog.pkl`
Web side	`web/src/lib/meal-targets.ts`, `web/src/app/api/generate-plan/route.ts`
Evidence	`models/reports/meal_recommender_rolling.json`, `meal_recommender_rolling.png`, `meal_catalog.json`
Data (offline only)	`ml-service/data/foodcom/RAW_recipes.csv`, `RAW_interactions.csv`
Configuration / dependencies	`ml-service/.env` (`ML_SERVICE_API_KEY`), `ml-service/.venv`

Command (PowerShell, inside `ml-service`):

```
.venv\Scripts\python.exe demo_models.py 2
```

Rebuild (do not run live): `.venv\Scripts\python.exe -m training.build_meal_catalog`. Evaluation: `-m training.evaluate_meal_rolling`.

6. Live demonstration procedure

Step	What to do and show
1. Show the targets	Open `web/src/lib/meal-targets.ts` lines 69–106. Explain BMR → TDEE → goal adjustment → per-meal split.
2. Open the model code	`app/routers/recommend.py`: `RESTRICTION_FLAGS` (line 31), `SLOT_COURSES` (47), `candidates_for` (74), calorie band logic (120–130).
3. Show the data safety	Open `training/data_foodcom.py` line 257, `dietary_flags`. Say: "flags come from ingredients, because the dataset's own vegetarian tag was wrong on 631 recipes."
4. Run the model	`.venv\Scripts\python.exe demo_models.py 2`
5. Show the output	Point to the daily target (2094 kcal), then the recipes per slot: each inside its calorie band, all vegetarian, ranked by likes.
6. Explain the result	Show the halal case returning 0 recipes. Open `meal_recommender_rolling.png`: popularity (blue) is above ALS (orange) in 4 of 6 years.

7. What code to show

Where	What it does	Why it matters
`recommend.py` 61–72	Splits restrictions into supported and unsupported	Safety: unverifiable restrictions get nothing
`recommend.py` 74–87	Slot, diet, protein and exclusion filters	The core selection
`recommend.py` 117–130	±25% band, widened to ±50%, top N	Calorie fitting
`data_foodcom.py` 257–275	Ingredient-based dietary flags	Where correctness comes from
`build_meal_catalog.py` 55–75	Popularity window and sort order	The ranking
`generate-plan/route.ts` 680	`enforceRecipeMeals`	LLM output cannot bypass the recommender

8. Expected demonstration output

```
--- STEP BEFORE MODEL 2 (Next.js, web/src/lib/meal-targets.ts) ---
BMR (Mifflin-St Jeor) = 1689 kcal   TDEE = BMR x 1.55 = 2618 kcal
Daily target (weight loss, -20%) = 2094 kcal

--- OUTPUT (HTTP 200) ---
method=popularity_last_365d  version=meal-popularity-v1  restrictions=['vegetarian']

  BREAKFAST  band 393-655 kcal  widened=False
    - the best belgian waffles                  636.0 kcal   13.5 g protein   83 likes last year
    - 5 minute vegan pancakes                   444.8 kcal   10.0 g protein   41 likes last year
    - basic pancake mix                         454.4 kcal   13.5 g protein   16 likes last year
  LUNCH  band 550-916 kcal  widened=False
    - easy stove top macaroni cheese            590.7 kcal   23.0 g protein   92 likes last year
    - olive garden fettuccine alfredo           880.3 kcal   19.0 g protein    9 likes last year
    - ruth s chris sweet potato casserole       783.1 kcal    9.0 g protein    8 likes last year
  DINNER  band 471-785 kcal  widened=False
    - yummiest ever baked mac and cheese        709.7 kcal   30.0 g protein    6 likes last year
    - easy eggplant parmesan                    650.1 kcal   28.0 g protein    4 likes last year
    - easy no boil pasta bake                   645.8 kcal   29.5 g protein    4 likes last year
  SNACK  band 157-261 kcal  widened=False
    - easy moist banana blueberry muffins       193.9 kcal    2.5 g protein   28 likes last year
    - champinones al ajillo garlic fried mushrooms 193.8 kcal  3.5 g protein   6 likes last year
    - very moist banana nut bread               226.7 kcal    2.5 g protein    4 likes last year

--- SAFETY CASE: a restriction the flags cannot verify ---
  halal -> 0 meals returned, reason: cannot verify recipes against: halal
```

How to verify it worked: HTTP 200, `stub: false`, every kcal value inside its band. Results are deterministic, so the same recipes appear on every run.

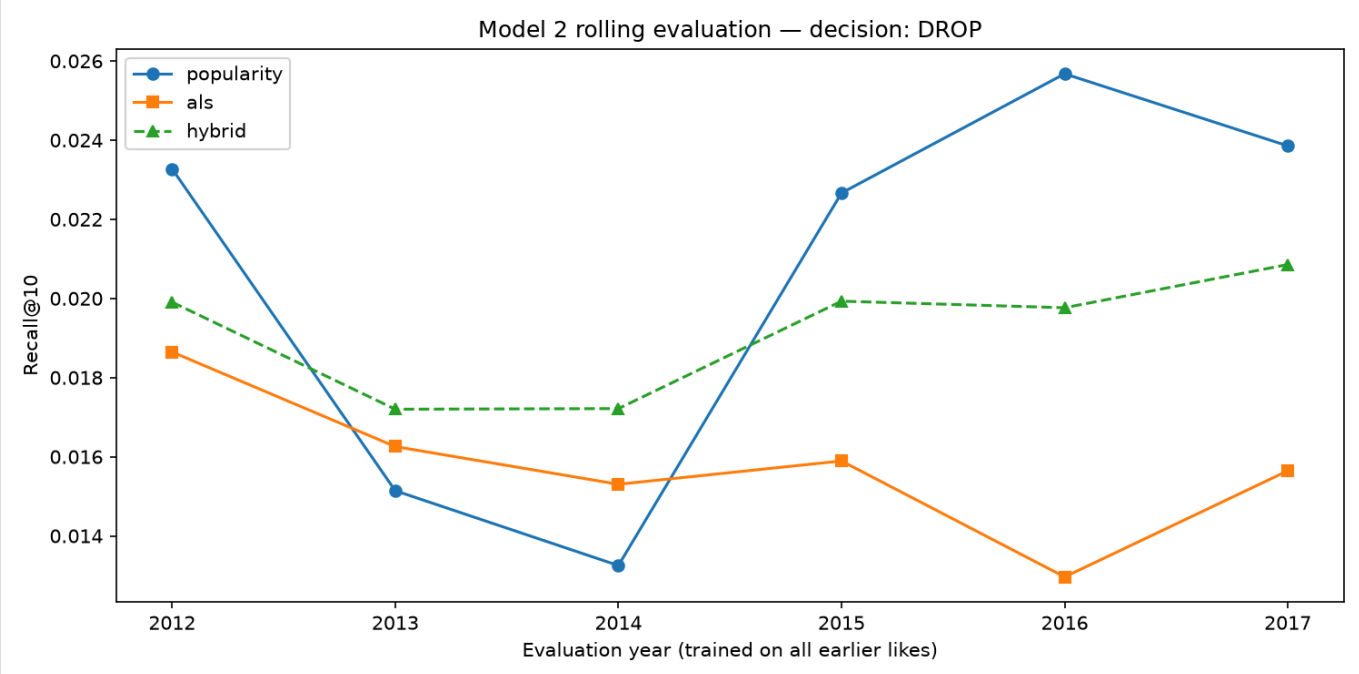


Figure 2: `models/reports/meal_recommender_rolling.png`. Recall@10 per evaluation year. Popularity (blue) vs ALS (orange) vs exploratory hybrid (green). Pooled: popularity 0.0197, ALS 0.0164.

9. Viva explanation

30 seconds

"Model 2 replaces LLM-invented meals with real recipes. Given a calorie target per meal and the user's dietary restrictions, it picks the most-liked recent Food.com recipes that fit the course, the restriction and the calorie band. The plan generator is then forced to use those recipes."

1 minute

"Model 2 replaces LLM-invented meals with real recipes. Given a calorie target per meal and the user's dietary restrictions, it picks the most-liked recent Food.com recipes that fit the course, the restriction and the calorie band. I trained an ALS collaborative filter and compared it with popularity in a rolling evaluation over six years and twenty thousand user-years. Popularity won on Recall@10, 0.0197 against 0.0164, with a confidence interval entirely below zero, so I shipped the simpler method. Dietary safety comes from checking ingredient lists, which I audited by hand. If a restriction like halal cannot be verified, it returns no recipes instead of risky ones."

Technical (if asked deeper)

- **Implicit feedback:** a 4–5 star rating counts as a like.
- **ALS:** factors 64, alpha 2, reg 1, tuned on a validation split.
- **Rolling evaluation:** for each year Y from 2012 to 2017, train on likes before Y and test on likes in Y. Popularity is recomputed from the prior 365 days.
- **Statistics:** pooled Recall@10, 95% CI from a bootstrap that resamples users. Decision rules were fixed before running.
- **Safety audit:** vegetarian flag 50/50 correct on a fresh random sample; regression tests in `tests/test_meal_data.py`.

10. Likely questions from teachers

Question	Answer
Why did you use this model?	Popularity measurably beat a trained ALS model on held-out future data. Shipping the weaker model would be dishonest.
What is the input?	Per-meal calorie targets (with optional protein floors) and dietary restrictions.
What is the output?	Ranked real recipes per meal slot with calories, protein, time and like counts.
Why this architecture?	Chosen by evaluation, not assumption: popularity won the rolling test. Rule-based filters give guaranteed dietary and calorie compliance, which a learned ranker cannot guarantee.
What preprocessing is performed?	Parse the nutrition column, build ingredient-based flags, map course tags, remove condiments and sauces, keep 20–2000 kcal, apply the popularity window.
What is it learning?	Nothing at serving time. The ranking is aggregated like counts. The ML work is in the evaluation that chose it.
How is it evaluated? Metrics?	Recall@10: of the recipes a user liked in the test year, the share that appeared in the top 10. Values are low (~2%) because there are ~98,000 recipes to choose from.
Limitations?	No personalisation (`user_id` unused); data ends in 2018 and is mostly American food; calories are per serving as listed by recipe authors; halal, kosher, pescatarian and low sodium unsupported; popular dishes skew towards heavy comfort food.

How is the output passed on?	`/api/generate-plan` puts recipes in the LLM prompt, then `enforceRecipeMeals` guarantees the saved diet plan uses them.
What if the input is invalid?	Bad slot, 0 or >3000 kcal, or no meals → HTTP 422. Unsupported restriction → empty list with a reason. Catalog not loaded → `stub: true`. Service down → Next.js wrapper returns an empty result and plan generation continues.
How is it different from simple image processing / a simple lookup?	No images are involved. Compared with a plain lookup, it combines verified dietary flags, calorie-band fitting and popularity ranking, and was chosen through a statistical comparison with a trained recommender.

11. Complete End-to-End Demo Flow

> **Important:** Model 1's calorie number is not an input to Model 2. Both models start from the same user profile. The flow below is how the code actually works.

```
                      +-- Cardio session (HR, minutes) --> MODEL 1 --> calories -->
cardio_logs.calories_burned
  User profile --------+   (uses age, sex, weight)
  (age, sex, height,   |
   weight, activity,   +-- meal-targets.ts (Mifflin-St Jeor) --> per-meal kcal targets
   goal, restrictions)        + dietary restrictions --> MODEL 2 --> real recipes
                                                                          |
                                          Groq LLaMA diet plan  <------+
                                          enforceRecipeMeals --> diet_plan made only of real
recipes
```

Stage	Explain before	Open	Run / show	Explain after
0. Setup	"Next.js app + Python ML service; each model is its own endpoint with a fallback."	`CLAUDE.md` model table	`cd ml-service`	n/a
1. Profile	"One demo user: 22, male, 175 cm, 70 kg, moderately active, weight loss, vegetarian."	`demo_models.py` lines 28–32	n/a	"Both models use this profile."
2. Model 1	"Estimate calories for a 30-minute cardio session at 150 bpm."	`energy.py`, `calorie.py`	`demo_models.py 1`	13.84 → 7.83 → 234.9 kcal; fallback reasons
3. Model 1 output	"This is what the Cardio tab saves."	`CardioTab.tsx` line 222	Figure 1	"Error ~1.3 kcal/min, measured on held-out people."
4. Targets	"Model 2 does not use Model 1's number. Targets come from Mifflin-St Jeor, whose activity factor already includes exercise."	`meal-targets.ts`	n/a	2094 kcal → 524 / 733 / 628 / 209
5. Model 2	"Find real recipes for each slot."	`recommend.py`	`demo_models.py 2`	Recipes in band, vegetarian, ranked by likes; halal → none
6. Final result	"The LLM writes the plan, but code enforces the recipes."	`generate-plan/route.ts` line 680	Figure 2	"The final result: a logged session with evidence-based calories, and a diet plan built only from real recipes."

Run both models in one go:

```
.venv\Scripts\python.exe demo_models.py
```

Optional browser view: `.venv\Scripts\python.exe -m uvicorn app.main:app --port 8000`, open `http://127.0.0.1:8000/docs`, click Authorize and paste the key. This view was not tested in preparation, so try it once beforehand. The script is the safer option because it needs no server.

Presentation Script

Model 1: Session Energy Expenditure

1. "First, this model is responsible for estimating the calories a user burns in a cardio session, using their heart rate instead of a fixed table."
2. "The input to the model is the session's average heart rate, duration and workout type, plus the user's age, sex and weight. It is validated by a Pydantic schema first."
3. "The model processes the input by applying the Keytel 2005 heart-rate equation, then a linear calibration, 0.1091 plus 0.5579 times the raw value, which I fitted on lab calorimetry data. If heart rate is missing, the workout is not cardio, or the heart rate is outside 83.5 to 182.7 bpm, it falls back to a MET table and returns the reason."
4. "The expected output is calories and calories per minute, the method used, and the expected error of 1.29 kcal per minute."
5. "Now I will run `demo_models.py 1`."
6. "As you can see, the model produced 13.84 kcal per minute from the raw equation, calibrated to 7.83, which is 234.9 kcal for 30 minutes, with method heart_rate. The fallback cases show that it reports why whenever it cannot use heart rate. This plot shows the calibrated predictions sitting on the diagonal against the lab measurements."
7. "This output is then passed to the Cardio tab, which saves it as the session's calories burned in the cardio log."

Model 2: Meal Recommender

1. "First, this model is responsible for recommending real recipes for each meal, so the diet plan does not contain meals or calories invented by the LLM."
2. "The input to the model is a calorie target for each meal slot plus the user's dietary restrictions. The targets come from the Mifflin-St Jeor formula: 2094 kcal a day for this user, split across breakfast, lunch, dinner and a snack."
3. "The model processes the input by filtering 98,325 Food.com recipes by course, by ingredient-checked dietary flags and by a ±25% calorie band, then ranking what is left by how many people liked each recipe in the last year of the data. I tested an ALS collaborative filter against this, and popularity won on Recall@10, so the simpler method ships."
4. "The expected output is a ranked list of recipes per meal, each with calories, protein, cooking time and like counts."
5. "Now I will run `demo_models.py 2`."
6. "As you can see, the model produced three vegetarian recipes per slot, each inside its calorie band. For example, easy eggplant parmesan at 650 kcal for a 628 kcal dinner target. When I ask for halal, which the flags cannot verify, it returns nothing instead of guessing."
7. "This output is then passed to the plan generator. The LLM writes the plan from these recipes, and `enforceRecipeMeals` replaces any meal that is not one of them, so the final diet plan contains only real recipes with real calories."

Pre-presentation checklist

- Run `.venv\Scripts\python.exe demo_models.py` once on the presentation laptop and confirm `calorie=True recommender=True`.
- Confirm `ml-service/models/calorie_energy.json` and `meal_catalog.pkl` exist.
- Have `calorie_hr_calibration.png` and `meal_recommender_rolling.png` open in tabs.
- Do not open `ml-service/.env` on screen.
- Increase the terminal font size so the output is readable from the back of the room.